

华东师范大学数据科学与工程学院实验报告

课程名称: 数据库管理系统	年级: 2024	上机实践时间: 2026.04.17
指导教师: 周烜	姓名: 孙育泉	
上机实践名称: 问卷系统 II - 题库驱动架构与版本控制	学号: 10234900421	

目录

I 实验任务	1
II 使用环境	1
III 实验过程	2
III.1 Vibe Coding 大致过程	2
III.2 数据库设计演进	2
III.3 API 说明	3
III.4 核心技术与算法思路实现细节	4
III.4.1 1. 不可变 (Immutable) 控制与版本衍生实现	4
III.4.2 2. Aggregation 聚合框架打破文档壁垒提取全局统计	4
III.4.3 3. 安全防护 (Security Barrier) 与自动修补引擎	5
IV 测试结果	5
V 总结	6

I 实验任务

在系统第一阶段上线并运行后, 根据用户使用反馈, 原有的“题目嵌套在问卷中强绑定”的设计模式暴露出严重的灵活性问题. 因此, 本阶段 (第二阶段) 核心任务是将系统架构升级为题库驱动体系 (Question-Bank-Centric Architecture) .

新增核心需求如下 (参考自 `demand.md`):

- (1) 题目复用与共享: 用户可以独立保存常用题目, 从已有的独立题库导入到新问卷中, 甚至将题目无缝共享给团队其他群体成员.
- (2) 历史与版本控制: 更新或修改题目时不影响已经发布的历史问卷. 同一个宏观概念下修改会保留为独立可溯源的并行快照新版本.
- (3) 题目引用追踪: 要求系统能够反向查询某个具体的题库题目当前正在被哪些生效的问卷所引用装载.
- (4) 跨问卷全局统计: 针对通用类题目 (如“您的年龄”被跨组使用多次), 系统提供汇聚全部历史问卷的全局可视化与统计面板.

II 使用环境

- 后端数据库: MongoDB
- 编程语言: Python
- 后端框架: FastAPI
- 前端框架: Vue.js (搭配 Element Plus 拦截配置)
- 包管理器: uv + npm
- AI 工具: Gemini (Antigravity Agent)

III 实验过程

III.1 Vibe Coding 大致过程

[1] **Prompt** Review it and devise your own task list or refer to the task list in the markdown file. and modify this project to meet all the demands in the markdonw file

[1] **Result** 配置依赖项与环境, 理解系统向题库为中心 (Question-Bank) 过度并提纯分离架构的核心研发理念并拆分逻辑任务

[2] **Prompt** go ahead, but first config environment

[2] **Result** 搭建并验证 MongoDB (v7) 数据库环境、梳理 Python uv 依赖空间并在工作区建立稳定运行的服务流.

[3] **Prompt** now the demand.md changes, please read the new demands and and identify whether there are some features that are not yet implement and implement them

[3] **Result** 着手后端重构, 完成独立题库集合创建及不可变快照版本控制逻辑.通过 MongoDB Aggregation 开发提取高难度全局统计数据的算法, 并配套重写后端响应路由及所有附带关联的 Pydantic 模型.

[4] **Prompt** is there a button which user can choose a question to add to the bank when clicked?

[4] **Result** 补充 Vue 开发, 完成 QuestionBank.vue 引擎与 SurveyBuilder.vue 路由模块衔接操作, 实现在新问卷中直接导入题目的弹窗以及在题目局部卡片上的版本解绑保留机制.开发前端全局统计看板挂载于 UI.

[5] **Prompt** when i want to submit the question, it pop a window with only a cross warning, and nothing happens

[5] **Result** 追踪排查系统内部极度隐蔽的 HTTP 422 状态校验失效及 UI 级渲染缺失 BUG.定位到前端因拦截器 (request.js) 解开数据包而引起的未定义问题同时, 修复为保障数据安全性而采取的由于漏传必须字段 original_q_id 引致的大型拒绝.使用服务端强制自动追溯补充的方法解决.

[6] **Prompt** 我现在在从题库导入里面还是看不到任何题目.还有, 我希望每个问题在创建的时候, 有个按钮, 选择是否共享题目.

[6] **Result** 对自动提取逻辑做出业务性强化, 在问卷编辑器右侧植入入库前独立 Share 拨动按键以及后端相关属性配置支持; 修复由于新数据模型上线与 Axios 断层带来的数据为空表空转问题.

[7] **Prompt** 我希望在管理界面增加一个功能, 可以直接查看题库, 类似 demand.md 第八个要求.

[7] **Result** 在控制台 Dashboard 框架中补充专门的题库总览及宏观全局数据通道与按键, 进一步打通各级接口, 完成最终功能汇集.

III.2 数据库设计演进

为了解耦题目与问卷, 并完美应对只读防篡改溯源以及高效跨盘统计需求, 我们在第一阶段的集合之上演化了架构.

(1) users 集合: 不变.(维持注册信息)

(2) 新增核心 question_bank 集合: 全独立题库.它的条目是系统最小知识单元快照.

- _id: ObjectId, 代表题目的具体唯一快照版本.
- original_q_id: String, 由于一个题目修改会产生多个历史分裂项, 所有同源版本共享这个标识.是跨问卷统计的数据汇聚点.

- version: Integer, 快照版本代数 (递增) .
- is_shared: Boolean, 是否对外全服公开该单元.
- parent_version_id: 指向衍生的前置 _id 节点, 支持画出修撰树状图.
- 常规数据: type, title, options, constraints...

(3) 修改后的 surveys 集合:

- questions 中增加了由系统维护的弱外挂键: question_bank_id (指向到唯一的 _id 快照实体) .
- 虽然引用外壳, 但为了防止 MongoDB Join 开销且同时保护已建立历史不受损 (Immutable), 我们依然采用全冗余文档存储所有题目详情在问卷内部.这保障了即便是原题在题库中发生物理删除, 问卷历史依然不受影响.

(4) 修改后的 responses 集合:

- 在 answers 内核数组追加至高维度标记字段 question_bank_original_id.这一巧妙设置直接连通所有的答卷单元和宏观通用题目, 是进行高速无损聚合提取的最本质先决条件.

III.3 API 说明

这里仅摘录并说明本阶段由于拓展新增涉及题库的特殊核心 API.

(1) 按需获取公共及内部题库

- 接口地址: GET /api/questions
- 访问权限: 需登录
- 传入参数: is_shared=true (可选) 获取公共区.
- 返回格式:

json

```

1 [
2   {
3     "id": "69f21ab ... ",
4     "original_q_id": "90abcd ... ",
5     "version": 1,
6     "is_shared": false,
7     "title": "请问你的出生年份是",
8     "type": "single"
9   }
10 ]

```

(2) 基于原本快照衍生并推送修订新祖干版本

- 接口地址: POST /api/questions/{q_id}/versions
- 功能描述: 传入当前版本的改动物质, 触发系统继承 original_q_id, 分裂为高代分支并返回新派生实体的属性.
- 请求格式与新建题型同理.

(3) 检索版本历程树 (Version Tree Histroy)

- 接口地址: GET /api/questions/{original_q_id}/history
- 返回围绕相同概念 original_q_id 的所有过往递进历程记录数组.

(4) 定位问卷引用的依存明细

- 接口地址: GET /api/questions/{q_id}/dependencies
- 功能: 提供当前该版本的快照实际上正存在于系统中哪些运行且有效的物理问卷记录当中.返回包含对应 Survey 名称及存活态数据的清单.

(5) 利用引擎获取跨层全局全数据统筹

- 接口地址: GET /api/stats/question/{original_q_id}
- 请求格式: 无 (Params ID directly)
- 响应结果 (自动根据单选、数字类统计归类完毕的集成图表信息):

json

```

1 {
2   "original_q_id": "abc ... ",
3   "title": "跨级总设问: 你最近喝咖啡吗? ",
4   "type": "single",
5   "total_responses": 113,
6   "options_count": { "是": 90, "否": 23 }
7 }

```

III.4 核心技术与算法思路实现细节

在本阶任务实践时, 单纯 CRUD 绝不能达到题目系统的关联需求, 为此在算法思路和代码层次有诸多细节:

III.4.1 1. 不可变 (Immutable) 控制与版本衍生实现

当发布了的问卷与题目有牵引关系时, 任何原位置的暴力 Update 操作后果都是灾难性的. 我们在 Python 后端使用了类似 Git 的分裂机制:

python

```

1 @router.post("/{q_id}/versions", response_model=QuestionBankResponse)
2 async def create_new_version(q_id: str, question: QuestionBankCreate):
3     # 查找老版本的原身以继承家族原始基因 (original_q_id)
4     old_q = await db.question_bank.find_one({"_id": ObjectId(q_id)})
5
6     # 获取属于该基因流的最大版本号
7     max_ver_doc = await db.question_bank.find_one(
8         {"original_q_id": old_q["original_q_id"],
9          sort=[("version", -1)]
10    )
11     new_version = max_ver_doc["version"] + 1
12
13     doc = question.model_dump()
14     doc["original_q_id"] = old_q["original_q_id"]
15     doc["version"] = new_version
16     doc["parent_version_id"] = q_id # 指明从哪里产生分裂, 构造网状关系
17
18     result = await db.question_bank.insert_one(doc)
19     return ...

```

通过分裂操作, 任何一版被装载的历史都在被安全的锁定保护之中, 不同团队成员即便套用同一套题库作答也可独自向自己分支产生修订.

III.4.2 2. Aggregation 聚合框架打破文档壁垒提取全局统计

随着调查表不断填充, MongoDB 中海量的嵌套在层层 Document 中的 answers 字段变成孤岛. 查询“关于全校年龄收集情况的混合统计”若采用传统的先 Fetch 再 Python for 循环统计, 会立刻遭遇大规模高密度的内存阻断或极大延迟. 在这里, 纯粹交给 MongoDB 底层 C++ 原生执行流高效完成:

python

```

1 pipeline = [
2     # 拆分! 把每个文档对象中的每道题打散做平摊拆解
3     {"$unwind": "$answers"},

```

```

4
5 # 瞄准射击! 运用新增加的原始宏号实现高速度高精度命中
6 {"$match": {"answers.question_bank_original_id": original_q_id}},
7
8 # 剔除无法贡献统计的完全废弃脏数据和无效流
9 {"$match": {
10     "answers.value": {"$ne": None, "$not": {"$type": "array", "$size": 0}},
11     "$ne": ""}
12 },
13
14 # 终极聚合! 利用 facet 等同态特性分类分发按选项累计数据
15 {"$facet": {
16     "total": [{"count": "count"}],
17     "options_distribution": [
18         # 自动遍历所有值进行 count 步长计算整合
19         {"$group": {"_id": "$answers.value", "count": {"$sum": 1}}}
20     ]
21 }
22 ]
23 stats_result = await db.responses.aggregate(pipeline).to_list(length=1)

```

此种 Aggregation Pipeline 将计算阻力阻拦在服务器之外, 无论跨度覆盖了十张问卷或是万份填写册, 时间复杂度和输出极为优异平滑.

III.4.3 3. 安全防护 (Security Barrier) 与自动修补引擎

我们在实现跨卷统计必须要求每次作答的数据流追加 original_q_id, 但前端的 Payload 极不稳定易受跨站编造修改 (如用户通过抓包删去了此必填关键参数). 这在 Vibe Coding 试验时遭遇了 422 引致前后端界面大规模瘫痪. 解决方案在于是我们放弃了对 Client 侧参数的依赖幻想, 转由在后端路由自主重拼溯源获取并强制封存记录, 这构成了一道坚实的安全铁闸:

python

```

1 # 获取来自问卷主表的依赖题库缓存标识, 并不依赖于被提交侧的信息
2 qb_ids = [ObjectId(q["question_bank_id"]) ... ]
3 # 自主联结 MongoDB 本地源追溯
4 cursor = db.question_bank.find({"_id": {"$in": qb_ids}})
5 qb_map = {str(doc["_id"]): doc.get("original_q_id") for doc in qb_docs}
6
7 valid_answers = []
8 for ans in response_data.answers:
9     ans_dict = ans.model_dump()
10     ... # 这里自行赋值拼合最纯净安全无误的原始键再入库, 拦截所有注入
11     ans_dict["question_bank_original_id"] = qb_map.get(qb_id_str)

```

IV 测试结果

针对第二阶段的所有更新, 进行交互流验证:

在创建问卷界面, 成功新增“从题库导入”与基于原始单题设计的“是否共享至共享厅”开关, 测试其导入数据后原逻辑 (如强校验、跳转依赖限制) 依然完好不被剥离. 此时可以在工作台中观测到刚刚补入的题库入口“ 管理题库中心”, 进入便能使用独立的看板分析界面, 这赋予了产品对极长运行周期的深度运营特性. 在选择“版本历史”后, 弹窗完美浮现修订时间、版本迭代递增, 进而能以指定基点实现任意历史时刻覆盖复活. 再通过点击任意题目“全局统计”, 即触发 MongoDB 流汇集生成统计表或百分饼图视图, 无论被关联调用至多少杂项文件内皆一览无余地将该单品的完整历史表现具象.

V 总结

本阶段（阶段 II）针对在线问卷系统的架构进行了深刻地剥离梳理.为了克服传统强绑定导致的“改一处崩全局”困境,引入并实现了以题库为第一维度的驱动框架(Bank-Centric Architecture).系统参考了源代码管理领域的“版本分层控制思想 (Version Control)”和“唯加操作不变策略 (Immutable Record)”完成了不同问卷内同一主题各自演进的数据安全阻隔墙.并有效且高效地施展了文档型数据库在应对大范围解构重组时发挥的深层 API——利用 Aggregation Pipeline 多通道数据聚集处理能力攻陷了单源向跨源转移统合的信息屏障.在面对全环境组件 (Vuex/Axios) 联动带来的多维异常捕获隐蔽困局时,利用安全回追校验及错误层流规范有效封堵了潜在崩溃节点.总体而言,该版本系统完美兑现了所需求的团队协作、共促生产力的完整设想.